

Module-4.

Bayesian learning.

Fundamentals of Bayes Theorem:-

① ~~Prior~~

Introduction to probability based learning:-

Bayes theorem. works on 3 kinds of probability.

- ① Prior probability
- ② Likelihood probability
- ③ ~~prior~~ posterior probability.

① Prior-

- It is a general probability of an uncertain event before an observation is seen or some evidence is collected.
- It is initial probability that is believed before any new information is collected.

② Likelihood

- It is the relative probability of the observation occurring for each class or sampling density for evidence given the hypothesis.
- It is stated as
$$P(\text{Evidence} | \text{Hypothesis})$$

③ Posterior

- It is updated or revised probability of an event taking into account the observations from the training data.

$$P(\text{Hypothesis} | \text{Evidence})$$

- posterior probability = prior probability + new evidence.

Classification using Bayes model:-

① MAP hypothesis.

- Given a set of candidate hypothesis, the hypothesis which has the max value is considered as most probable hypothesis or max probable hypothesis (MAP).

② ML hypothesis:

- Given a set of candidate hypotheses, if every hypothesis is equally probable, only $P(E|h)$ is used to find most probable hypothesis.
- This hypothesis that gives maximum likelihood is called as ML hypothesis.

Example:

Consider a boy who has a volleyball tournament on the next day, but today he feels sick. It is unusual that there is only a 40% chance he would fall sick since he is a healthy boy. Now, Find the probability of the boy participating in the tournament. The boy is very much interested in volleyball, so there is a 90% probability that he would participate in tournaments and 20% that he will fall sick given that he participates in the tournament.

① Prior probability $P(h)$.

② $P(E|h)$.

③ $P(h|E)$.

Sol: $P(\text{Boy participating tournament}) = 90\%$.

$P(\text{Boy is sick}) = 40\%$.

$P(\text{He is sick} | \text{Boy participate tournament}) = 20\%$.

Hence the probability of boy participating in tournament given that he is sick is defined as

$$P(h|E) = \frac{P(E|h) \cdot P(h)}{P(E)}$$

$$\frac{P(\text{Boy participating tournament})}{\text{He is sick}} = \frac{P(\text{He is sick} | \text{Boy participating tournament}) \times P(\text{Boy participates})}{P(\text{He is sick})}$$

$$= \frac{0.2 \times 0.9}{0.4}$$

$$= 0.45 //$$

Naive Bayes algorithm.

~~Naive based~~

~~Naive~~ Bayes Theorem is a supervised learning algorithm with binary class or multiclass classification algorithm which relies on the principle of Bayes theorem.

Steps:-

- ① Compute the prior probability for the target class $P(H)$
- ② Compute frequency matrix and likelihood probability ($P(E|H)$) for each of the feature.
- ③ Use Bayes theorem for calculating the probability of all hypotheses.

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

- ④ Use map hypothesis, h_{MAP} to classify the test object to the hypothesis with the highest probability.

$$h_{MAP} = \max$$

Example: Use temp data.

Step 1:- Calculate the prior

The prior probability for the target $Joboffer = \text{Yes} \rightarrow 7$
 $Joboffer = \text{No} \rightarrow 3$

Prior probability for $Joboffer = \text{Yes} \Rightarrow \frac{7}{10}$

Prior probability for $Joboffer = \text{No} \Rightarrow \frac{3}{10}$

Step 2: Calculate the frequency matrix & likelihood for all features.

Attribute: CGPA

frequency matrix

CGPA	Joboffer = Yes	Joboffer = No
29	3	1
28	4	0
28	0	2
Total	<u>7</u>	<u>3</u>

Likelihood probability.

CGPA	$P(\text{Job offer} = \text{Yes})$	$P(\text{Job offer} = \text{No})$
≥ 9	$P(\text{CGPA} \geq 9 \text{Job} = \text{Yes}) = 3/7$	$P(\text{CGPA} \geq 9 \text{Job} = \text{No}) = 1/3$
≥ 8	$P(\text{CGPA} \geq 8 \text{Job} = \text{Yes}) = 4/7$	$P(\text{CGPA} \geq 8 \text{Job} = \text{No}) = 0/3$
< 8	$P(\text{CGPA} < 8 \text{Job} = \text{Yes}) = 0/7$	$P(\text{CGPA} < 8 \text{Job} = \text{No}) = 2/3$

Attribute: Interactiveness.

Frequency matrix	Yes	No
Yes	5	1
No	2	2
Total	7	3

Likelihood probability

I	$P(\text{Job} = \text{Yes})$	$P(\text{Job} = \text{No})$
Yes	$5/7$	$1/3$
No	$2/7$	$2/3$

Attribute: PK

PK	Yes	No
Very good	2	0
Good	4	1
Average	1	2
Total	7	3

PK	Yes	No
VG	$2/7$	$0/3$
G	$4/7$	$1/3$
A	$1/7$	$2/3$

Attribute: CS

CS	Yes	No
Good	4	1
moderate	3	0
Poor	0	2
Total	7	3

CS	Yes	No
Good	$4/7$	$1/3$
moderate	$3/7$	$0/3$
Poor	$0/7$	$2/3$

For the instances, $(CGPA \geq 9 | \text{JB} = \text{Yes})$, $PK = \text{Average}$, $IN = \text{Yes}$, $CS = \text{Good}$.

↑
Test Instances.

↓
For this instance predict whether the student will get a job or not.

Step

Finding the probability $JB = \text{Yes}$ & $JB = \text{No}$.

$$P(JB = \text{Yes} | \text{Test data}) = \frac{P(CGPA \geq 9 | JB = \text{Yes}) \cdot P(IN = \text{Yes} | JB = \text{Yes}) \cdot P(PK = \text{Avg} | JB = \text{Yes}) \cdot P(CS = \text{Good} | JB = \text{Yes})}{P(JB = \text{Yes})}$$

$$= \frac{\frac{3}{7} \times \frac{5}{7} \times \frac{1}{7} \times \frac{4}{7} \times \frac{7}{10}}{10}$$

$$= \cancel{0.0357} = 0.0175$$

$\frac{P(E|H) \cdot P(H)}{P(E)}$
 $P(E)$
↑
Ignored

$$P(JB = \text{No} | \text{Test data}) = \frac{P(CGPA \geq 9 | JB = \text{No}) \cdot P(IN = \text{Yes} | JB = \text{No}) \cdot P(PK = \text{Avg} | JB = \text{No}) \cdot P(CS = \text{Good} | JB = \text{No})}{P(JB = \text{No})}$$

$$= \frac{\frac{1}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} \times \frac{2}{10}}{10}$$

$$= \cancel{0.0022} = 0.0074$$

Step 4:- Applying MAP hypothesis, the student will get job offer because

$P(JB = \text{Yes} | \text{Test data}) = 0.0175$ which is maximum.

∴ Student will get the job.

Zero probability error.

The zero probability error can be solved ~~by~~ by applying a Laplace correction.

In Laplace correction method values are scaled to 1000.

By putting $\frac{0}{300}$ as $\frac{1}{300}$ $\rightarrow$ only for values that gives zero.
negligible value.

eg:- consider the test instance.

(CGPA ≥ 8 , I=Yes, PK=A, CS=Good)

Probability:-

$$P(JB=Yes | \text{Test data}) = \frac{4}{7} \times \frac{5}{7} \times \frac{1}{7} \times \frac{4}{7} \times \frac{7}{10} = 0.0233$$

$$P(JB=No | \text{Test data}) = \frac{0}{3} \times \frac{1}{3} \times \frac{2}{3} \times \frac{1}{3} \times \frac{3}{10} = 0$$

Scale - CGPA

≥ 9	300/700	100/300
≥ 8	400/700	0/300
≥ 8	0/700	200/300
		$\downarrow$
		101/303
		1/303
		201/303

$$\Rightarrow P(JB=Yes | \text{Test data}) = 0.0233$$

$$P(JB=No | \text{Test data}) = \frac{1}{303} \times \frac{100}{300} \times \frac{200}{300} \times \frac{100}{300} \times \frac{303}{1003} = 0.02325$$

Bayes optimal Classifier:-

It is a probabilistic model, it uses Bayes theorem to find the most probable classification for a new instance given the training data by combining the predictions of all posterior hypothesis.

This is different from maximum A Posteriori (MAP) hypothesis, h_{MAP} which chooses the maximum probable hypothesis is the most probable hypothesis.

Q. Given the hypothesis space with 4 hypothesis h_1, h_2, h_3 & h_4 .
Determine if the patient is diagnosed with COVID +ve or COVID -ve
using Naive Bayes optimal classifier.

$P(h_i T)$	$P(\text{COVID +ve} h_i)$	$P(\text{COVID -ve} h_i)$
0.3	0	1
0.1	1	0
0.2	1	0
0.1	1	0

Solⁿ.

Step 1:- Summation of.

$$\Sigma P(\text{COVID -ve} | h_i) \cdot P(h_i | T) = 0.3 \times 1 = 0.3$$

$$\Sigma P(\text{COVID +ve} | h_i) \cdot P(h_i | T) = 0.1 \times 1 + 0.2 \times 1 + 0.1 \times 1 = 0.4 (\text{max})$$

$\therefore$ maximum is {COVID +ve, COVID -ve}

$$P(h_i | T) = \text{COVID +ve}$$

Thus this algorithm diagnoses the new instance to COVID +ve

Naive Bayes Algorithm for continuous attributes:-

2 ways to predict with Naive Bayes algorithm for

① Discrete continuous feature to discrete features.

② Apply Normal or Gaussian distribution for continuous feature.

Gaussian Naive Bayes Algorithm:

SNO.	CGPA	Interactiveness	T.offers
1	9.5	Yes	Yes
2	8.2	No	Yes
3	9.3	No	No
4	7.6	No	No
5	8.4	Yes	Yes
6	9.1	Yes	Yes
7	7.5	Yes	No
8	9.6	No	Yes
9	8.6	Yes	Yes
10	8.3	Yes	Yes

The training dataset T consist of 10 data instance with attributes as shown above. The target variable is JobOffer.

Test Data: To predict CGPA = 8.5 & Interactiveness = Yes will get the Job or Not.

Step 1: Compute the prior probability for target class:

$$(Yes, No) = (7, 3)$$

$$\text{Prior-Probability } (JB = Yes) = \frac{7}{10} \quad \text{prior-probability } (JB = No) = \frac{3}{10}$$

Step 2: Compute frequency matrix & likelihood probability for each attribute.

CGPA

→ CGPA is continuous attribute hence likelihood probability is calculated using ~~the~~ either Gaussian or normal distribution.

• Gaussian distribution for continuous features:

$$P(X_i = x_k | c_j) = g(x_k, \mu_{ij}, \sigma_{ij}) \quad \text{--- (1)}$$

Hence the normal distribution is given by

$$P(X_i = x_k | c_j) = \frac{1}{\sigma_{ij} \sqrt{2\pi}} e^{-\frac{(x_k - \mu_{ij})^2}{2\sigma_{ij}^2}} \quad \text{--- (2)}$$

X_i = Continuous attribute in the given dataset.

x_k = the value of attribute

c_j = denotes j th class of the target feature.

μ_{ij} = mean value of continuous attribute.

σ_{ij} = standard deviation of continuous attribute.

$$X_i = \text{CGPA}$$

$$x_k = 8.5$$

$$c_j = \text{"JobOffer = Yes"}$$

$$c_j = \text{"JobOffer = No"}$$

$$\mu_{ij} = \mu_{\text{CGPA} = \text{Yes}} = 8.8142$$

$$\mu_{ij} = \mu_{\text{CGPA} = \text{No}} = 8.1383$$

$$\sigma_{ij} = \sigma_{\text{CGPA} = \text{Yes}} = 0.581$$

$$\sigma_{ij} = \sigma_{\text{CGPA} = \text{No}} = 1.01159$$

Interactiveness:-

- Compute the likelihood probability for interactiveness

① Frequency matrix

I	Yes	No
Yes	5	1
No	2	2
Total	7	3

② Likelihood

I	$P(JB=Yes)$	$P(JB=No)$
Yes	$5/7$	$1/3$
No	$2/7$	$2/3$

Step 3: Calculate Bayes theorem

Test Data = Data (CGPA = 8.5 ; I = Yes)

Generate 2 hypothesis (JB = Yes ; No)

$$\textcircled{1} P(JB=Yes | \text{Test Data}) = P(CGPA=8.5 | JB=Yes) \times P(I=Yes | JB=Yes) \times P(JB=Yes)$$

$$P(X_{CGPA}=8.5 | JB=Yes) =$$

$$= \frac{1}{\sigma_{ij} \times \sqrt{2\pi}} e^{-\frac{(x_{ij} - \mu_{ij})^2}{2\sigma_{ij}^2}}$$

$$P(CGPA=8.5 | JB=Yes) = g(x_i - 8.5, \mu_{ij} = 8.814, \sigma_{ij} = 0.581)$$

$$= \frac{1}{0.581 \times \sqrt{2\pi}} e^{-\frac{(8.5 - 8.814)^2}{2 \times 0.581^2}} = 0.5942$$

$$\Rightarrow P(JB=Yes | \text{Test data}) = 0.5942 \times \frac{5}{7} \times \frac{7}{10} = 0.297$$

$$\textcircled{2} P(JB=No | \text{Test data}) = P(CGPA=8.5 | JB=No) \times P(I=Yes | JB=No) \times P(JB=No)$$

$$P(X \in PA) = 8.5 \quad (JB = No)$$

$$\Rightarrow \mu_R = 8.5 \quad \mu_{ij} = 8.1383 \quad \sigma_j = 1.01159$$

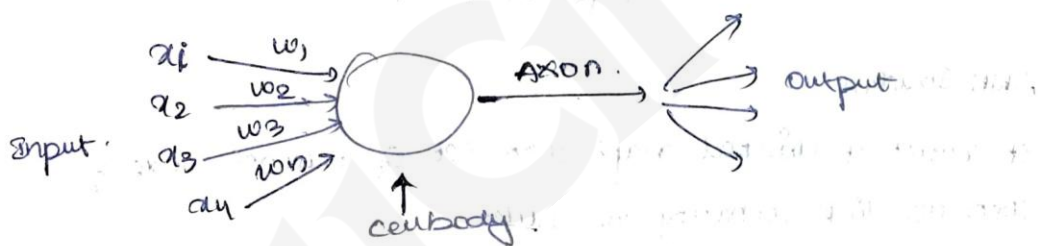
$$\Rightarrow \frac{1}{1.01159 \times \sqrt{2\pi}} e^{-\frac{(8.5 - 8.1383)^2}{2 \times 1.01159^2}} = 0.37116$$

$$P(JB = No | \text{Test Data}) = 0.37116 \times \frac{1}{2} \times \frac{2}{10} = 0.037116$$

Step 4: Applying MAP hypothesis, The student will get ~~JB~~ Toboffel because $P(JB = Yes | \text{Test data}) = 0.297$ is maximum.

Artificial Neural Networks:-

- They are like biological neurons, which are called as nodes.
- Nodes can receive 1 or more dendrites & process it.



Simple model of an Artificial Neurons:

- The first mathematical model of a biological neuron was introduced by McCulloch.

- ① Receiving weighted inputs from other neurons
- ② Applying an activation function to determine whether the neuron fires.

→ The input $x_1, x_2, \dots, x_n$ are multiplied by corresponding weights $w_1, w_2, \dots, w_n$.

→ Summed up to form a Net-sum.

$$\text{Net-sum} = \sum x_i \cdot w_i$$

→ Activation function is typically binary step function.

Output is 1 if $\text{sum Net-sum} \geq \text{threshold } \theta$.

Output is 0 if the $\text{Net-sum} < \text{threshold } \theta$.

→ This tells that artificial neurons process i/p produce o/p in neural networks.

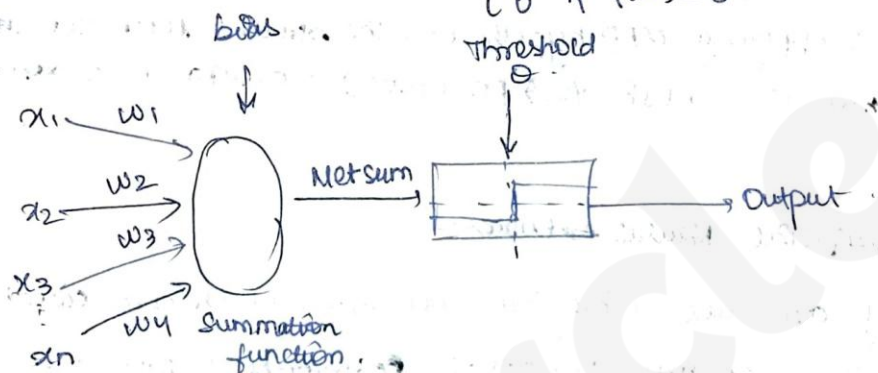


- The activation function is a binary step functions which outputs a value 1 if net-sum is above the threshold value θ , and 0 if net-sum is below the threshold value θ .

$\therefore$ Activation function is applied to Net-sum

$$f(x) = \text{Activation function (Net-sum)}$$

$$\text{Output of a neuron } y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}$$



Representation of Neurons

ANN Structure:-

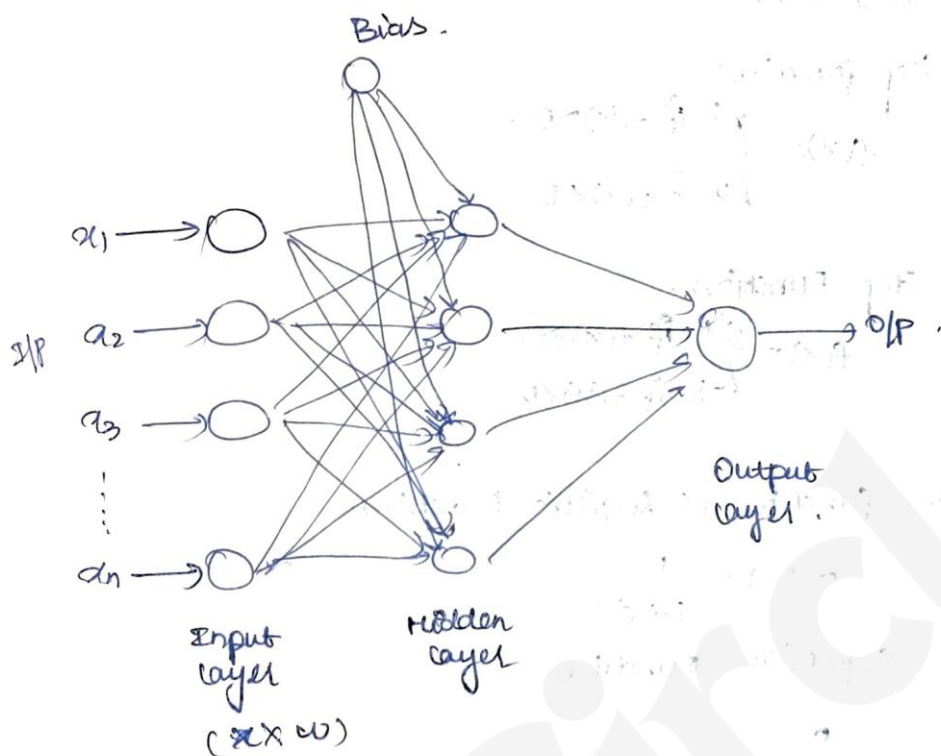
- It consists of directed graph with set of neurons nodes & connection link connecting the nodes.
- The network has 3 important layers:
 - Input layer
 - Hidden layer
 - Output layer

① Input layer:

- Input layer receives i/p and passes it to nodes in the hidden layer.
- They adjust the connecting nodes from i/p layer to hidden layer are associated with weights.
- These computing nodes perform computation based on the i/p information received.
- If the weighted sum of the i/p to the neurons is above the threshold of activation of the neurons then neurons fires.

• then the activation function map weighted i/p sum to a non-linear o/p value.

• the mode in o/p layer gives the o/p as single value.



Structure of Artificial neural networks.

Activation function:

- Activation function is a mathematical function ~~also~~ associated with each neurons in the neural network.
- It maps the input signal to the output signal.
- It is a function that decides whether neurons is fired or not.
- These function normalizes the output value of each neuron between $0 \leq 1$ or between -1 or $+1$.

List of activate function:-

① Linear / Identity function:

$$f(x) = x \quad \forall x.$$

② Binary Step function:

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ 0 & \text{if } f(x) < 0 \end{cases}$$

③ Bipolar Step Function.

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ -1 & \text{if } f(x) < 0 \end{cases}$$

④ Sigmoidal function or logistic Function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

⑤ Bipolar Sigmoidal Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}}$$

⑥ Ramp function.

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases}$$

⑦ Tanh - Hyperbolic Tangent Function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

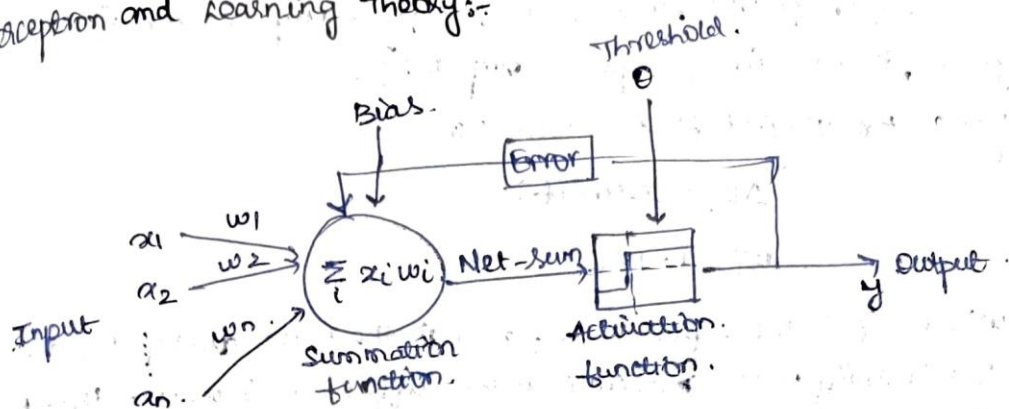
⑧ ReLU - Rectified Linear Unit Function

$$\sigma(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

⑨ Softmax Function

$$S(x_i) = \frac{e^{x_i}}{\sum_{j=0}^K e^{x_j}} \quad \text{where } i=0, \dots, K.$$

Perceptron and Learning Theory:-



epoch $\rightarrow$ will tell how many times we are training (iterations).

$\uparrow$ Perceptron model $\uparrow$

The modified neuron model receives a set of inputs $x_1, x_2, \dots, x_n$, their associated weights $w_1, w_2, \dots, w_n$ and a bias. ~~The activation function~~

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad \text{--- (1)}$$

$$f(x) = \text{Activation function (Net-sum + bias)} \quad \text{--- (2)}$$

The activation function is a binary step function which outputs a value 1, if $f(x)$ is above the threshold value θ , and a 0 if $f(x)$ is below the threshold value θ .

$$y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad \text{--- (3)}$$

Perceptron Algorithm:-

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

① Compute the weighted sum by multiplying the inputs with the weights and add the products.

② Apply the activation function on the weighted sum:

$$y = \text{step}(x_1 w_1 + x_2 w_2 - \theta)$$

③ If the sum is above the threshold value, output the value as positive else output the value as negative.

④ Calculate the error by subtracting the estimated output $y_{\text{estimated}}$ from the desired output y_{desired} :

$$\text{error } e(f) = y_{\text{desired}} - y_{\text{estimated}}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y].

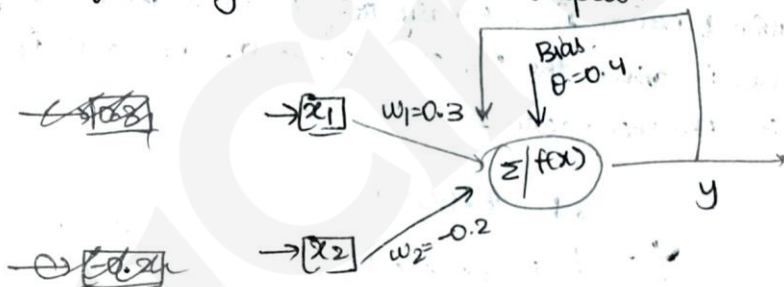
⑤ Update the weights if there is an error:

$$\Delta w_i = \alpha \cdot x_i \cdot e(t)$$

$$w_i = w_i + \Delta w_i$$

where x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

Q: Consider a perceptron to represent the boolean function AND with the initial weights $w_1=0.3$, $w_2=-0.2$, learning rate $\alpha=0.2$ and bias $\theta=0.4$ as shown in Figure below. The activation function used here is the step function $f(x)$ which gives the output value as binary, i.e., 0 or 1. If value of $f(x)$ is greater than or equal to 0, it outputs 1 or else it outputs 0. Design a perceptron that performs the Boolean Function AND and update the weights until the Boolean function gives the desired output.



Sol:-

2 input and gate.

q1

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND gate multiplication

Epoch 1

x_1	x_2	Ydes	Yest	error
0	0	0	$\text{step}((0 \times 0.3 + 0 \times -0.2) - 0.4) = -0.4 \Rightarrow 0$	0
0	1	0	$\text{step}((0 \times 0.3 + 1 \times (-0.2)) - 0.4) = -0.6 \Rightarrow 0$	0
1	0	0	$\text{step}((1 \times 0.3 + 0 \times (-0.2)) - 0.4) = -0.1 \Rightarrow 0$	0
1	1	1	$\text{step}((1 \times 0.3 + 1 \times (-0.2)) - 0.4) = -0.3 \Rightarrow 0$	1

$$\Delta w_i = \alpha \cdot e(t) \cdot x_i$$

$$\alpha = 0.2$$

~~For~~

For; $\alpha = 0.2, e(t) = 1, x_1 = 1, w_1 = 0.3$

$$\Delta w_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = 0.3 + 0.2$$

$$w_1 = 0.5$$

For; $\alpha = 0.2, e(t) = 1, x_2 = 1, w_2 = 0.2$

$$\Delta w_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = 0.2 + 0.2$$

$$w_2 = 0.4$$

$\therefore$ The changed weights are 0.5 & 0.4.

Exch 2:

$$w_1 = 0.5, w_2 = 0$$

x_1	x_2	y_{des}	y_{test}	<u>error</u>	<u>w_1</u>	<u>w_2</u>
0	0	0	$\text{step}((0 \times 0.5 + 0 \times 0) - 0.4) = -0.4 \Rightarrow 0$	0	0.5	0
0	1	0	$\text{step}((0 \times 0.5 + 1 \times 0) - 0.4) = -0.4 \Rightarrow 0$	0	0.5	0
1	0	0	$\text{step}((1 \times 0.5 + 0 \times 0) - 0.4) = 0.1 \Rightarrow 1$	-1	0.3	0
1	1	1	$\text{step}((1 \times 0.5 + 1 \times 0) - 0.4) = 0.1 \Rightarrow 1$	0	0.5	0.2

For, $\alpha = 0.2, e(t) = -1, x_1 = 1, w_1 = 0.5$

$$\Delta w_1 = 0.2 \times (-1) \times (1)$$

$$= -0.2$$

$$w_1 = 0.5 - 0.2$$

$$w_1 = 0.3$$

For, $\alpha = 0.2, e(t) = -1, x_2 = 0, w_2 = 0$

$$\Delta w_2 = 0.2 \times (-1) \times (0)$$

$$= 0$$

$$w_2 = 0 - 0$$

$$= 0$$

Exch 3:

$$w_1 = 0.3, w_2 = 0$$

x_1	x_2	y_{des}
0	0	0
0	1	0
1	0	0
1	1	1

Test

$$\text{step}((0 \times 0.3 + 0 \times 0) - 0.4) = -0.4$$

$$\text{step}((0 \times 0.3 + 1 \times 0) - 0.4) = -$$

For, $\alpha = 0.2, e(t) = 1, x_1 = 1, w_1 = 0.3$

$$\Delta w_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = 0.3 + 0.2$$

$$= 0.5$$

For, $\alpha = 0.2, e(t) = 1, x_2 = 1, w_2 = 0$

$$\Delta w_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = 0 + 0.2$$

$$= 0.2$$

Epoch 3:

0.5, 0.2

x_1	x_2	y_{des}	Yes	error	w_1	w_2
0	0	0	$\text{step}(0 \times 0.5 + 0 \times 0.2) - 0.4 = -0.4 \Rightarrow 0$	0	0.5	0.2
0	1	0	$\text{step}(0 \times 0.5 + 1 \times 0.2) - 0.4 = -0.2 \Rightarrow 0$	0	0.5	0.2
1	0	0	$\text{step}(1 \times 0.5 + 0 \times 0.2) - 0.4 = 0.1 \Rightarrow 1$	-1	0.3	0.2
1	1	1	$\text{step}(1 \times 0.3 + 1 \times 0.2) - 0.4 = 0.1 \Rightarrow 1$	0	0.3	0.2

For, $\alpha = 0.2$, $e(t) = -1$, $x_1 = 1$, $w_1 = 0.5$

$$\Delta w_1 = 0.2 \times -1 \times 1 = -0.2$$

$$w_1 = 0.5 - 0.2$$

$$w_1 = 0.3$$

For, $\alpha = 0.2$, $e(t) = -1$, $x_2 = 0$, $w_2 = 0.2$

$$\Delta w_2 = 0.2 \times -1 \times 0$$

$$= 0$$

$$w_2 = 0.2 + 0$$

$$w_2 = 0.2$$

Epoch 4:

0.3, 0.2

x_1	x_2	y_{des}	Yes	error
0	0	0	$\text{step}(0 \times 0.3 + 0 \times 0.2) - 0.4 = -0.4 \Rightarrow 0$	0
0	1	0	$\text{step}(0 \times 0.3 + 1 \times 0.2) - 0.4 = -0.2 \Rightarrow 0$	0
1	0	0	$\text{step}(1 \times 0.3 + 0 \times 0.2) - 0.4 = -0.1 \Rightarrow 0$	0
1	1	1	$\text{step}(1 \times 0.3 + 1 \times 0.2) - 0.4 = 0.1 \Rightarrow 1$	0

Since there is no error occurring in epoch 4, the weights $w_1 = 0.3$ & $w_2 = 0.2$ we get the desired outputs.

- It is observed that with 4 Epochs, the perceptron learns and the weights are updated to 0.3 and 0.2 with which the perceptron gives the desired output of a Boolean AND gate.

XOR Problem:-

- A perceptron model can solve all Boolean functions which are linearly separable.

↑
uses single layer perceptron model.

- But for XOR gate we have to use multi layer perceptron model.

↓
Some ~~outputs~~ ^{inputs} → Output.
different inputs → 1

↓
~~Deep layer~~
DNN.
ANN can be used.

Gradient Descent:- (2 marks)

- This is an optimization method to minimize the cost function by calculating the gradient (slope) of the errors with respect to weights.

- updating weights in the opposite direction of the gradient.

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \frac{\partial E}{\partial w} \rightarrow \text{where } \alpha \text{ learning rate.}$$

- using a learning rate to control the step size.

$\frac{\partial E}{\partial w}$: gradient of the error w.r.t the weight.

Delta Learning Rule:-

- Also known as Widrow-Hoff Rule or Adaline Rule.

- It's a method for updating weights in neural networks to reduce the error between the predicted and actual outputs.

$$\text{Training Error} = \frac{1}{2} \sum_{d \in T} (\text{Desired} - \text{Estimated})^2$$

where T is Training Dataset.

Advantages of ANN:-

- ① ANN can solve complex problems involving non-linear processes.
- ② ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
- ③ ANNs have a parallel processing capabilities and can predict in less time.
- ④ They have an ability to work with inadequate knowledge. It can even handle incomplete & noisy data.
- ⑤ They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations:-

- ① An ANN requires processing with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with large amount of data.
- ② They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each level.

Applications:-

- ① Real time application
- ② Business
- ③ Banking & Finance
- ④ Health care
- ⑤ Education
- ⑥ Other Engineering